

Technical Disclosure Commons

Defensive Publications Series

26 Sep 2024

Retrieval-Augmented Generation (RAG) Classification and Access Control

Assaf Namer

Robert Lagerström

Gopinath Balakrishnan

Brandon Maltzman

Follow this and additional works at: https://www.tdcommons.org/dpubs_series

Recommended Citation

Namer, Assaf; Lagerström, Robert; Balakrishnan, Gopinath; and Maltzman, Brandon, "Retrieval-Augmented Generation (RAG) Classification and Access Control", Technical Disclosure Commons, (September 26, 2024)

https://www.tdcommons.org/dpubs_series/7382



This work is licensed under a [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/).

This Article is brought to you for free and open access by Technical Disclosure Commons. It has been accepted for inclusion in Defensive Publications Series by an authorized administrator of Technical Disclosure Commons.

Retrieval-Augmented Generation (RAG) Classification and Access Control

ABSTRACT

Retrieval-augmented generation, a technique to improve the accuracy of responses of large language models (LLM), can result in an expanded attack surface for the LLM and an increased potential for exfiltration of sensitive data. This disclosure describes techniques to reduce the attack surface for LLMs that use retrieval-augmented generation. In a data upload phase, access permissions of data resources are recorded and quantified. Implicit permissions granted by the LLMs to different principals are assessed. If a data classification scheme is added to the data resources, a score can measure the risk of a prompt provided to the LLM, thus forestalling the possibility of users with lower classification level access to obtain data at higher classification levels. By analyzing relationships stored in a permissions database, hidden permissions can be exposed, which AI models might have otherwise inadvertently granted. This provides insight into potential security risks and enables cloud customers to govern and monitor the access to data used by LLM deployments.

KEYWORDS

- Retrieval augmented generation (RAG)
- Model grounding
- Large language model (LLM)
- Identity and access management (IAM)
- LLM agent
- LLM plug-in
- LLM extension
- LLM risk management
- Attack path
- Security operations (SecOps)
- AI security
- LLM security
- Data governance
- Security cloud

BACKGROUND

Retrieval-augmented generation (RAG), AI agents, and extensions are recent advances in artificial intelligence (AI) and machine learning (ML), particularly in the realm of natural language processing (NLP).

Large language models (LLM), while adept at generating plausible responses based on their training data, can struggle with maintaining up-to-date factual accuracy as their knowledge is limited to their training dataset. The need for RAG arises from the challenge of ensuring that LLMs produce responses that are not only coherent and contextually appropriate, but also factually accurate. RAG addresses accuracy in responses by combining an LLM with a retriever mechanism that fetches relevant information from a corpus of up-to-date online documents (usually with real-time information), such as online encyclopedias, news sources, and weather information. Some advantages of RAG include the following:

- Improves accuracy by incorporating external knowledge.
- Reduces hallucinations by grounding responses in factual data.
- Enables up-to-date information without full model retraining.
- Enhances context awareness for more relevant responses.
- Enables domain-specific knowledge integration.

Some advantages of agents include:

- Enable task decomposition for complex problem solving.
- Facilitate autonomous decision making and action taking.
- Improve adaptability to different environments and scenarios.
- Enable specialized capabilities through different roles.
- Enable multi-step reasoning and planning

Some advantages of extensions include:

- Expand LLM capabilities beyond text generation.
- Enable integration with external tools and application programming interfaces (APIs).
- Enable real-time data access and processing.
- Facilitate task-specific optimizations.
- Enable multimodal interactions, e.g., image processing and speech recognition.

Chatbots can be created that go beyond answering questions, e.g., taking actions and interacting with other systems to complete meaningful tasks.

While RAG, connectors, agents, plug-ins, and other related LLM extensions provide substantial value to LLM applications, they also expand the attack surface, increase the potential for data exfiltration, and can be used by an adversary to gain initial access to workloads of an organization. While the AI workload can be protected by a variety of controls [7], some controls can be invalidated or weakened when RAG is introduced.

A prompt sent to an LLM can result in a distinct RAG operation, which can ultimately lead to unauthorized access to sensitive data. These operations, which vary by mechanism used and location/type of data retrieved, can open up the organization to additional risk that is difficult to quantify. A user sending a prompt to an LLM might not have the appropriate permission to perform a specific action, but the user might bypass their assigned permissions and elevate their access by leveraging the inherent permissions of an LLM agent. An example of the problem of increased attack surface is illustrated in Fig. 1.

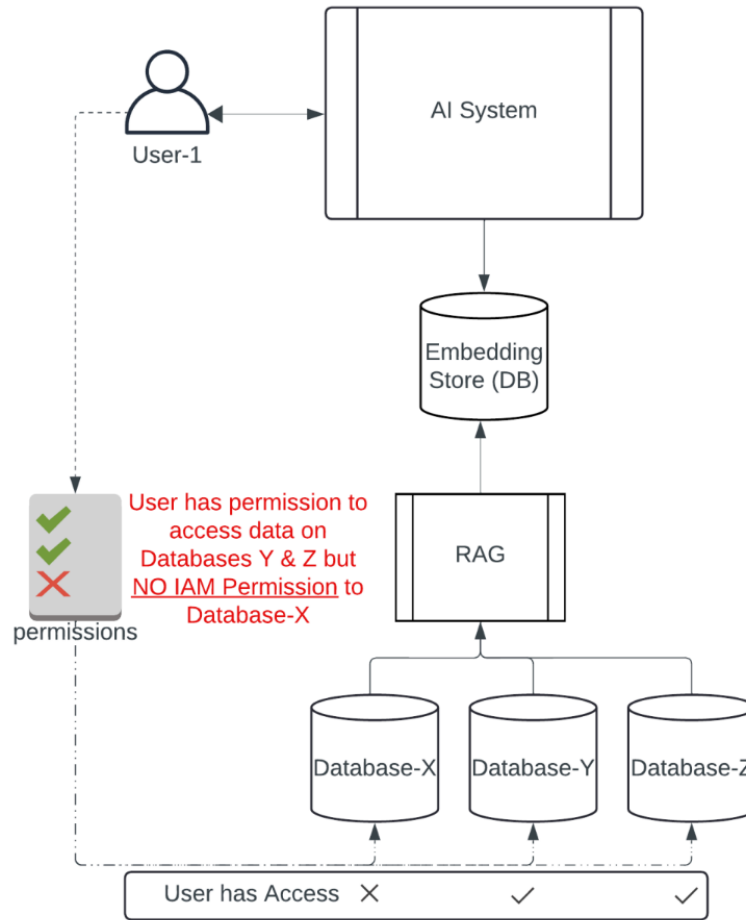


Fig. 1: A side-effect of RAG is an increased attack surface

In Fig. 1, User-1 does not have permission to access data in database-X per the identity and access management (IAM) system in place but does have permission to access data in other databases (i.e., Database-Y and Database-Z). However, using the AI system, which uses information from database-X via RAG, the user has implicit access to the database. The user can use prompt manipulation techniques to extract data from the database. Currently, there is no way to identify this implicit access to data from AI systems in real time. This problem is further amplified in large organizations with tens or hundreds of applications that use different types of agents as part of their services.

Enterprises classify data to categorize information based on its sensitivity and value in order to ensure appropriate handling, protection, and access control. These classifications typically range from public data to highly restricted information, with each level dictating specific security measures and usage policies. This practice is essential to maintain data confidentiality, comply with regulations, and protect sensitive information from unauthorized access or breaches. One regulation that requires data classification is the General Data Protection Regulation (GDPR). The problem described above of an increased attack surface is amplified when data from different classification levels is used by the RAG application.

DESCRIPTION

This disclosure describes techniques to reduce the attack surface for artificial intelligence (AI) models that use retrieval-augmented generation (RAG) to improve the accuracy of their responses. During a data upload phase, the access permissions of data resources are recorded and quantified. The implicit permissions granted by the AI models to different principals are assessed. Additionally, if a data classification scheme is added to the data resources, a score is introduced to evaluate the risk of the received prompt from a user to forestall the possibility of a user with lower classification level access to obtain data at a higher classification level. A permissions database, which can be a graph database, describes relationships between entities. By analyzing these relationships, hidden permissions that AI models inadvertently grant are exposed, providing valuable insight into potential security risks and enabling cloud customers to govern and monitor the access to data used by AI deployments.

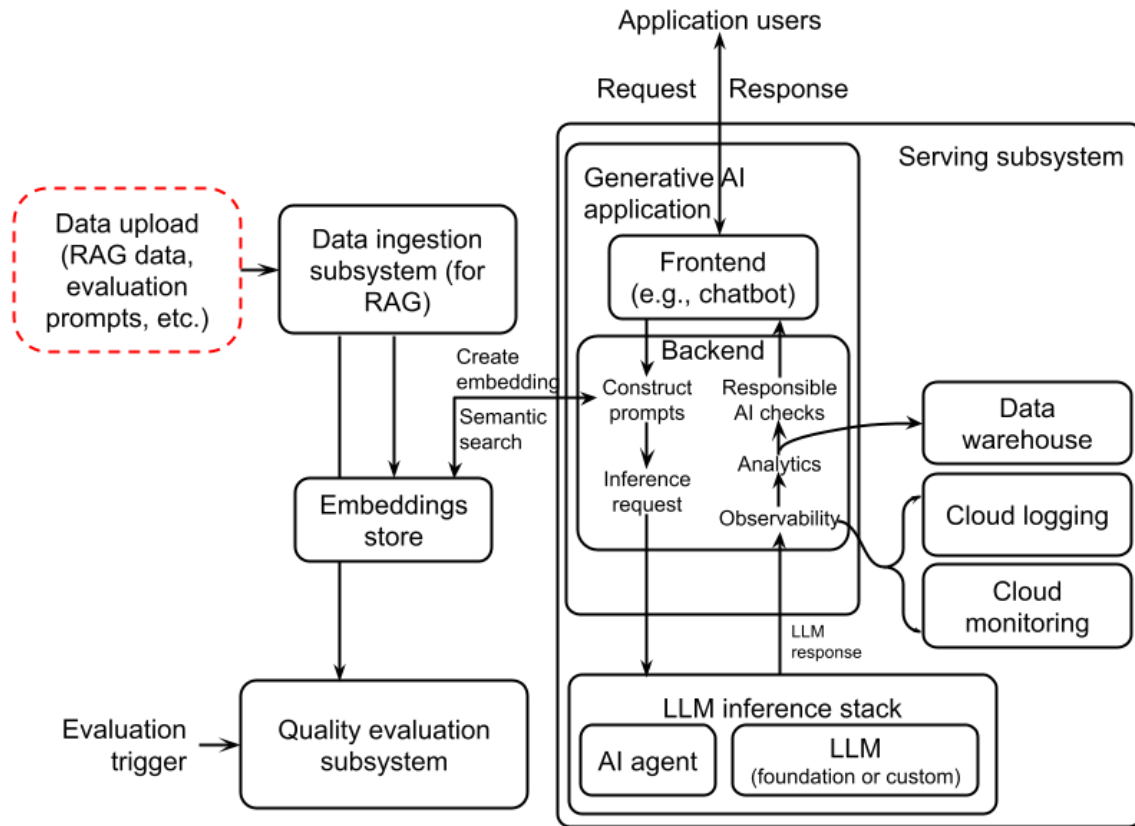


Fig. 2: RAG-based AI application using an AI agent [4]

Fig. 2, taken from [4], illustrates a typical architecture for RAG-based AI applications and model extensions using an AI agent. An embeddings store is leveraged for data that is retrieved in a data upload stage and ingested by a data ingestion subsystem. The data resources (e.g., files, database outputs) are vectorized and inserted into the embedding store database. Embeddings can be created from data using the techniques of, for example, [9].

This disclosure focuses on the portion of the architecture of Fig. 2 highlighted within the red box. During data upload and prior to data ingestion, a process to store and map permissions is performed on each data resource. The process, referred to as a RAG Authentication Index Process, is explained in greater detail below.

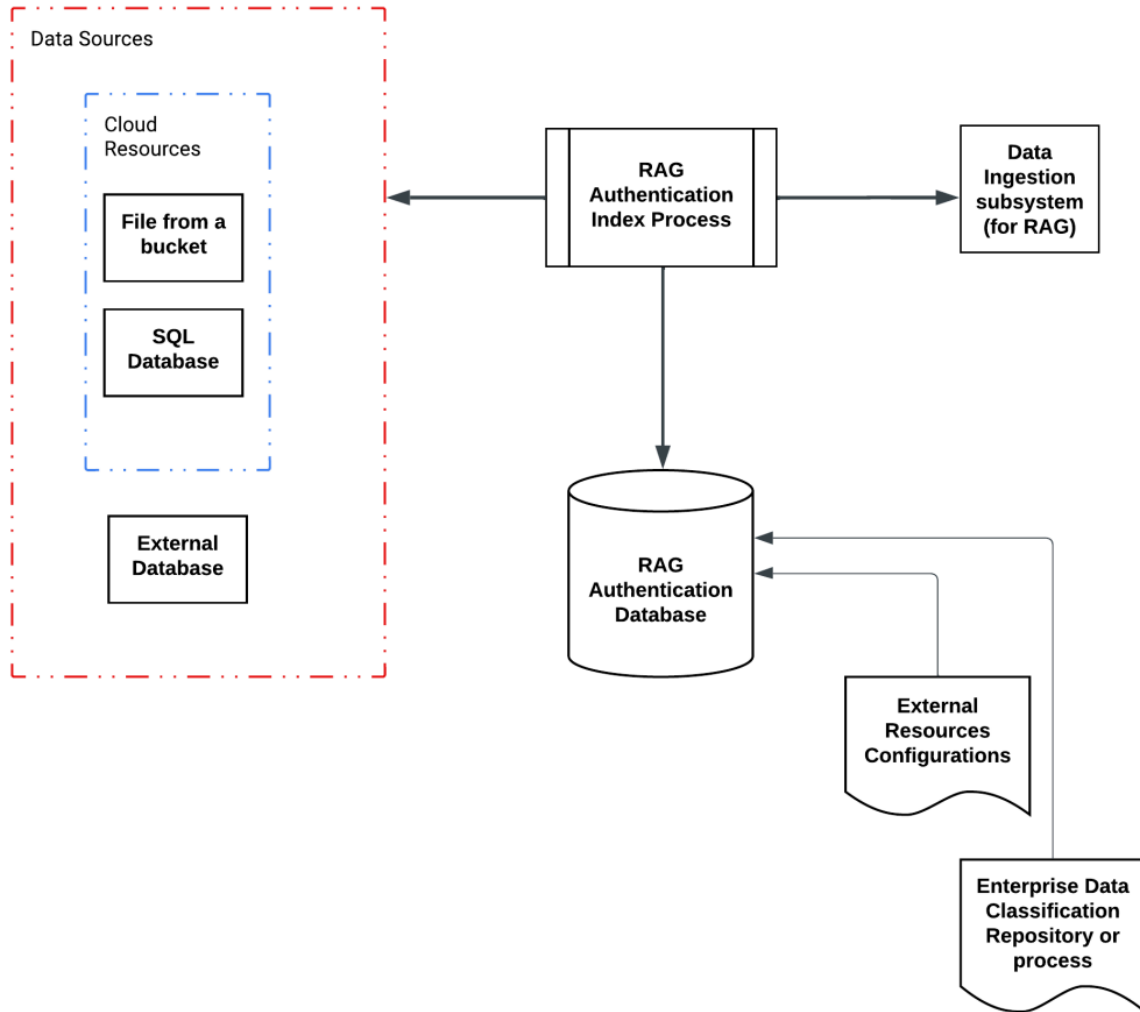


Fig. 3: RAG Authentication Index Process

Illustrated in Fig. 3, the RAG Authentication Index Process creates a table of resources, the associated authorizations required to access them, and their classification levels. For example, consider three data sources used to build the embedding store: two sources are cloud resources, such as a file from a bucket (e.g., a web object) and a cloud-based database (e.g., a SQL database), and one resource is a non-cloud (e.g., on-premises) external database.

The RAG Authentication Index Process inserts the resource identification attribute, associated permissions (e.g., users' and groups' access permissions), and the corresponding classification level into a RAG authentication database. The RAG authentication database can be

a graph database, a suitable option due to its ability to process queries about relationships among entities. The RAG Authentication Index Process captures resources hosted in the cloud and resources outside the cloud, such as third-party databases. Authorizations of external resources can be included in a separate configuration file that is accessible by the RAG Authentication Index Process. The separate configuration file can have a mapping of permissions between the external resources and a cloud principal, for example, an identity and access management (IAM) group. The mapping between the permissions of the external resource and the IAM group can be described in a human readable data-serialization language, such as YAML.

```
resources:
- name: Database-1
  type: database
  iam_id: IAM-Group-1
  Classification: top-secret

- name: Database-2
  type: database
  iam_id: IAM-Group-2
  Classification: Public
```

Fig. 4: YAML description of the permissions of an external resource mapped to an IAM group

For example, as illustrated in the YAML code of Fig. 4, Database-1 is a database resource hosted outside of the cloud. During data ingestion, the RAG Authentication Index Process attaches the IAM ID IAM-Group-1 to the Database-1 resource. This ultimately ensures that only principals that belong to that group are able to obtain responses that include data from different databases with a higher classification using the RAG AI application. Centralizing resources that are used to build the RAG application, such as databases, files, and web objects, and mapping those resources to IAM policies of the enterprise in a database, such as a graph

database, enables enterprises to control access to applications that use RAG while adhering to enterprise policies.

The RAG Authentication Index Process also utilizes the resource classification repository of the enterprise to add the data classification levels of various resources into the RAG authentication database. Thus, each entry in the RAG authentication database includes the data resource identification attribute, its permissions, and its classification level.

The classification levels determine the risk score of a user prompt. For example, if a user prompt triggered the retrieval of data via RAG, and the resource from where the data was pulled was classified as top secret, the risk score of the prompt request is higher than the risk score of a prompt request which triggered the retrieval of data via RAG from a resource classified as non-sensitive.

After data ingestion for RAG has been completed and the RAG authentication database is populated with the data resources and associated permissions and classification levels, the RAG authentication database is used to allow or deny responses from the RAG application, as illustrated below.

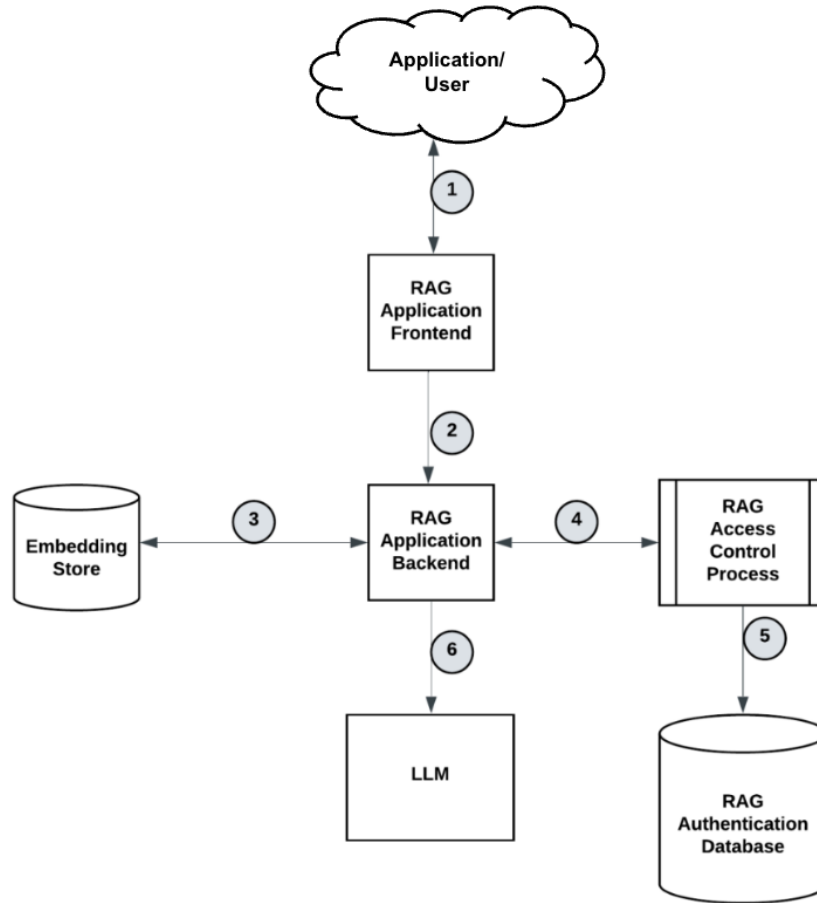


Fig. 5: Using RAG authentication database to control access to a RAG application response

Fig. 5 illustrates using the RAG authentication database to control (allow/deny) access to a RAG application's response. The numbered steps are explained below.

1. An authenticated application (user) sends a prompt to the RAG application frontend.
2. The application frontend sends the request to the backend.
3. The application backend accesses the embedding store and retrieves the following information:
 - a. Embeddings, which are vector representations of text chunks or documents;
 - b. Metadata, which can include
 - i. the original text chunk or document,

- ii. document identifiers,
 - iii. source information,
 - iv. timestamps,
 - v. relevance scores (if calculated during retrieval), etc.;
 - vi. Contextual information, which is metadata that provides context of the embedded text; etc.
4. The application backend does not construct and enrich the prompt with the relevant data from the embedding store yet; rather, it sends a query to the RAG access control process for permissions and classifications on the retrieved data by providing the metadata source information from the previous step.
 5. The RAG access control process uses the RAG authentication database to determine if the data resources to be used by the authenticated user is authorized. This can depend on the user's permission to the data sources per the data upload process. Evaluation of permissions and control of access can be done in multiple ways, for example, by leveraging a simple query using the path in a graph database as illustrated in Fig. 6 below.

```

MATCH (u:User {name:
$userName})-[:HAS_ROLE]->(r:Role)-[:HAS_PERMISSION]-
>(p:Permission)-[:ALLOWS_ACCESS]->(res:Resource {name:
$resourceName})

RETURN

EXISTS((u)-[:HAS_ROLE]->()-[:HAS_PERMISSION]->()-
[:ALLOWS_ACCESS]->(res)) AS hasAccess

```

Fig. 6: Example of permissions checking in a graph database

Additionally, using a graph database risk scoring query, the relationship between the user and the data resource is calculated based on the level of the data classification. Access to data from a resource with a high classification level will result in a higher risk score than access to data from a resource with a lower classification level. In a graph database, such scoring procedures can look like the operation illustrated in Fig. 7 below, where an AccessScore is calculated for a user Alice based on access to different data classification levels.

```
MATCH (u:User {name: 'Alice'})-[:HAS_CLEARANCE]-
>(c:ClearanceLevel),
      (d:DataResource {name: 'Financial Report'})-
[:HAS_CLASSIFICATION]->(cl:ClassificationLevel)

WITH u, d, c.value AS userClearance, cl.value AS
dataClassification

RETURN u.name AS User,
      d.name AS DataResource,
      dataClassification AS DataClassification,
      userClearance AS UserClearance,
CASE
  WHEN userClearance >= dataClassification THEN 10
  ELSE 10 - (dataClassification - userClearance) * 2
END AS AccessScore
```

Fig. 7: Sample query to calculate the risk score of a user based on the data classification of the resources used by the prompt response

6. Depending on the access evaluation request, the application backend forwards the prompt and the retrieved data to the LLM if the user of the application has access to all resources used in the RAG process and the prompt riskiness is below a predefined threshold. Otherwise, if the response requires access to unauthorized resources (i.e., resources the user is not authorized to access) or the riskiness is above the predefined threshold, a

response is delivered to the user that an answer cannot be provided, along with a reason why no answer was provided. Alternatively, a response can be crafted using only resources to which the user has access while ensuring the risk is below the predefined threshold. The final option would be to craft a response that redacts sensitive information as described below.

To elucidate on the concept of risk scores, the database can return risk scores for the relationships among entities based on their associated classifications. For example, if a data resource is classified as internal use only, it has a lower score than a data resource classified as restricted or top secret. Based on the combined risk score, the backend can implement logic to determine an appropriate response to the prompt. Optionally, rules can be set to block or redact information if the classification is above a certain threshold for a user. Visualization of the classifications can also be aggregated into a dashboard for insight and monitoring of AI-based applications.

In this manner, the described techniques ensure that users of AI deployments can only access data of sensitivity commensurate with their assigned classification level of access. In particular, a user (or application) that sends a prompt without having explicit access to the resource from which data is pulled is prevented from receiving a response that contains overly sensitive data relative to their role. Additionally, each request can be evaluated based on their risk score, and appropriate action can be taken based on predefined requirements.

The aforementioned techniques provide a superior way for enterprises to govern and apply restrictions on RAG applications by ensuring adherence to the enterprise access control and data classification policies. They enable cloud customers to adhere to various applicable regulations pertaining to data usage and exposure in AI applications, for example, as detailed in

[15]. The techniques enhance the flexibility with which customers can apply safety controls and better safeguard their critical data.

CONCLUSION

This disclosure describes techniques to reduce the attack surface for LLMs that use retrieval-augmented generation. In a data upload phase, access permissions of data resources are recorded and quantified. Implicit permissions granted by the LLMs to different principals are assessed. If a data classification scheme is added to the data resources, a score can measure the risk of a prompt provided to the LLM, thus forestalling the possibility of users with lower classification level access to obtain data at higher classification levels. By analyzing relationships stored in a permissions database, hidden permissions can be exposed, which AI models might have otherwise inadvertently granted. This provides insight into potential security risks and enables cloud customers to govern and monitor the access to data used by LLM deployments.

REFERENCES

1. “Retrieval augmented generation (RAG) in Azure AI search,” available online at <https://learn.microsoft.com/en-us/azure/search/retrieval-augmented-generation-overview> accessed on Sep. 21, 2024.
2. “What is RAG (retrieval-augmented generation)?” available online at <https://aws.amazon.com/what-is/retrieval-augmented-generation/> accessed on Sep. 21, 2024.
3. “Build enterprise gen AI apps with Google Cloud databases,” available online at <https://cloud.google.com/blog/products/ai-machine-learning/rag-with-databases-on-google-cloud> accessed on Sep. 21, 2024.

4. “Infrastructure for a RAG-capable generative AI application using Vertex AI,” available online at <https://cloud.google.com/architecture/rag-capable-gen-ai-app-using-vertex-ai> accessed on Sep. 21, 2024.
5. “Making AI helpful for everyone,” available online at <https://cloud.withgoogle.com/next/generative-ai#sessions> accessed on Sep. 21, 2024.
6. “Vertex AI agent builder: Build and deploy enterprise ready generative AI experiences,” available at <https://cloud.google.com/products/agent-builder?hl=en> accessed on Sep. 21, 2024.
7. “Best practices for securely deploying AI on Google cloud,” available online at <https://services.google.com/fh/files/misc/best-practices-for-securely-deploying-ai-on-google-cloud.pdf> accessed on Sep. 21, 2024.
8. “How someone can steal your large language model,” available online at <https://www.fuzzylabs.ai/blog-post/how-someone-can-steal-your-large-language-model> accessed on Sep. 21, 2024.
9. “Work with vector embeddings,” available online at [https://cloud.google.com/alloydb/docs/ai/work-with-embeddings#:~:text=Optionally%2C%20if%20you%20have%20the,See%20more%20code%20actions.&text=ALTER%20TABLE%20TABLE%20ADD%20COLUMN%20EMBEDDING_COLUMN%20vector\(%20DIMENSIONS%20\);,See%20more%20code%20actions.&text=UPDATE%20TABLE%20SET%20EMBEDDING_COLUMN%20=%20embedding,name%20of%20the%20embedding%20column](https://cloud.google.com/alloydb/docs/ai/work-with-embeddings#:~:text=Optionally%2C%20if%20you%20have%20the,See%20more%20code%20actions.&text=ALTER%20TABLE%20TABLE%20ADD%20COLUMN%20EMBEDDING_COLUMN%20vector(%20DIMENSIONS%20);,See%20more%20code%20actions.&text=UPDATE%20TABLE%20SET%20EMBEDDING_COLUMN%20=%20embedding,name%20of%20the%20embedding%20column) accessed on Sep. 21, 2024.

10. “Invest implications: forecast analysis: artificial intelligence software, 2023-2027, worldwide,” available online at <https://www.gartner.com/en/documents/4925331> accessed on Sep. 21, 2024.
11. Hemank Lowe, “Beyond hallucinations: how retrieval augmented generation and grounding anchor AI in reality,” available online at <https://www.linkedin.com/pulse/beyond-hallucinations-how-retrieval-augmented-generation-hemank-low-udmyc/> accessed on Sep. 21, 2024.
12. “Beyond RAG basics: strategies and best practices for implementing RAG,” available online at https://www.elastic.co/virtual-events/beyond-rag-basics?utm_campaign=Google-NB-Search-AMER-NA&utm_content=Rag-B&utm_source=google&utm_medium=cpc&device=c&utm_term=database%20rag&gad_source=1&gclid=Cj0KCQjwwuG1BhCnARIsAFWBUC2h_TY3tMnoxh4vmB6F-5uCX-6QX0cwiWOEh2I_NvThPrm2gefKZz4aAqYkEALw_wcB accessed on Sep. 21, 2024.
13. “What is a graph database?” available online at <https://neo4j.com/docs/getting-started/get-started-with-neo4j/graph-database/> accessed on Sep. 21, 2024.
14. “NIST special publication 800-53,” available online at <https://www.sailpoint.com/identity-library/nist-800-53> accessed on Sep. 21, 2024.
15. “Risk management process (Section 6.3.4)” in ISO 5338:2023, Information Technology — Artificial Intelligence — AI system life cycle processes available online at <https://cdn.standards.iteh.ai/samples/81118/8e53a667079f4d959e93119fdb84434/ISO-IEC-5338-2023.pdf> accessed on Sep. 21, 2024.